Azure Databricks — Comprehensive Hands-on Assessment

Scenario: Event Registration and Venue Management System

Part 1 — Dataset preparation

Run the following setup commands before starting the assessment.

Dataset 1 — Event registrations

```
%sql
CREATE OR REPLACE TABLE event_registrations_source (
    registration_id INT,
    event_id STRING,
    attendee_name STRING,
    registration_date STRING,
    ticket_type STRING,
    tickets INT,
    unit_price INT,
    status STRING,
    channel STRING
)
USING DELTA;
```

```
%sql
INSERT INTO event_registrations_source VALUES
(2001,'E01','Aarav','2026-10-01','Standard',2,500,'Confirmed','Web'),
(2002,'E02','Arnav','2026-10-02','Premium',1,1200,'Confirmed','App'),
(2003,'E03','Bhavna','2026-10-03','Standard',3,800,'Pending','Counter'),
(2004,'E04','Charan','2026-10-04','Standard',2,650,'Confirmed','Partner')
(2005,'E05','Diya','2026-10-05','VIP',1,1500,'Cancelled','Web'),
(2006,'E06','Eshan','2026-10-06','Standard',4,400,'Confirmed','App'),
(2007,'E01','Fiza','2026-10-07','Premium',1,1000,'Confirmed','App'),
(2008,'E02','Farah','2026-10-08','Premium',2,1200,'Pending','Web'),
(2009,'E03','Gauri','2026-10-09','Standard',1,800,'Confirmed','Counter'),
(2010,'E04','Harini','2026-10-10','Standard',3,650,'Confirmed','Web'),
```

```
(2011,'E05','Irfan','2026-10-11','VIP',2,1500,'Confirmed','Partner'),
(2012,'E06','Jiya','2026-10-12','Standard',1,400,'Cancelled','App'),
(2013,'E01','Kabir','2026-10-13','Standard',4,500,'Confirmed','Web'),
(2014,'E02','Lakshmi','2026-10-14','VIP',1,1800,'Confirmed','Counter'),
(2015,'E03','Mohan','2026-10-15','Standard',2,800,'Confirmed','Partner'),
(2016,'E04','Nisha','2026-10-16','Standard',1,650,'Pending','Web'),
(2017,'E05','Omkar','2026-10-17','VIP',3,1500,'Confirmed','App'),
(2018,'E06','Pari','2026-10-18','Standard',2,400,'Confirmed','Web'),
(2019,'E01','Qadir','2026-10-19','Standard',2,500,'Cancelled','Counter'),
(2020,'E02','Rohan','2026-10-20','Premium',3,1200,'Confirmed','Partner'),
(2021,'E03','Sana','2026-10-21','Premium',1,1300,'Confirmed','App'),
(2022,'E04','Tanvi','2026-10-22','Standard',2,650,'Confirmed',NULL),
(2023,'E05','Uday','2026-10-23','VIP',1,1500,'Pending','Web'),
(2024,'E06','Imran','2026-10-24','Standard',3,400,'Confirmed','App'),
(2025,'E01','Varsha','2026-10-25','Standard',1,500,'Confirmed','Web'),
(2026,'E02','Wasim','2026-10-26','Premium',2,1200,'Confirmed',NULL),
(2027,'E03','Yash','2026-10-27','Standard',0,800,'Pending','Counter'),
(2028,'E04','Zara','2026-10-28','Standard',2,0,'Confirmed','Web'),
(2029,'E05','Adil','2026-10-29','VIP',2,1500,'Confirmed','App'),
(2030,'E06','Bina','2026-10-30','Standard',1,400,'Pending','Counter'),
(2031,'E99','Chetan','2026-10-31','Standard',2,700,'Confirmed','Web'),
(2032,'E01','Danish','2026-11-01','Standard',3,500,'Confirmed','Partner')
```

Dataset 2 — Event master

```
%sql
CREATE OR REPLACE TABLE event_master (
    event_id STRING,
    event_name STRING,
    city STRING,
    category STRING,
    capacity INT
)
USING DELTA;

INSERT INTO event_master VALUES
('E01','Design Summit','Bengaluru','Technology',300),
('E02','Food Expo','Hyderabad','Culinary',250),
('E03','Art Fair','Pune','Culture',180),
('E04','Startup Meetup','Mumbai','Business',220),
('E05','Music Showcase','Chennai','Entertainment',400),
```

```
('E06','Book Festival','Delhi','Literature',150),
('E07','Photography Walk','Jaipur','Culture',90);
```

Dataset 3 — Working table

Use this table for all updates, deletes, and MERGE operations. Do not modify the source table until the incremental-loading section.

```
%sql
CREATE OR REPLACE TABLE event_registrations_work
USING DELTA
AS
SELECT * FROM event_registrations_source;
```

Dataset 4 — Incoming registrations

```
%sql
CREATE OR REPLACE TEMP VIEW incoming_registrations AS
SELECT *
FROM VALUES
(2002,'E02','Arnav','2026-10-02','Premium',1,1400,'Confirmed','App'),
(2008,'E02','Farah','2026-10-08','Premium',2,1200,'Confirmed','Web'),
(2024,'E06','Imran','2026-10-24','Standard',3,450,'Confirmed','App'),
(2033,'E07','Naina','2026-11-05','Standard',2,600,'Confirmed','Web'),
(2034,'E03','Omar','2026-11-05','Premium',1,900,'Pending','Partner'),
(2035,'E05','Leena','2026-11-06','VIP',2,1600,'Confirmed','App')
AS t(
    registration_id,event_id,attendee_name,
    registration_date,ticket_type,tickets,
    unit_price,status,channel
);
```

Part 2 — Assessment exercises

Section A — SQL fundamentals

1. Display all records from `event_registrations_source`.

2. Retrieve only registration ID, attendee name, event ID, and ticket price.
3. Find registrations where the status is `Confirmed`.
4. Retrieve registrations where the ticket price is greater than ₹1,000.
5. Find registrations with ticket types `Premium` or `VIP` using `IN`.
6. Retrieve registrations where ticket prices are between ₹500 and ₹1,200. Display the results in descending order of price.

Section B — Filtering, joins and aggregations

7. Display the distinct registration channels.
8. Find attendees whose names begin with `A` or contain the letter `a`, using `LIKE`.
9. Calculate the total number of tickets registered for each event.
10. Calculate the total revenue of confirmed registrations for each event using `tickets * unit_price`. Exclude invalid records with zero tickets or zero price.
11. Join `event_registrations_source` and `event_master` to display attendee name, event name, city, ticket type, and registration status.
12. Perform a left join and identify registrations whose event IDs do not exist in `event_master`. Also identify events with no registrations.

Section C — PySpark DataFrames

13. Load both Delta tables into separate PySpark DataFrames and display their schemas.
14. Select only `registration_id`, `attendee_name`, `tickets`, and `unit_price` using PySpark.
15. Filter records where the status is Confirmed and the number of tickets is greater than 2.
16. Create a new column named `total_amount` by multiplying `tickets` and `unit_price`.
17. Create a column named `booking_category` with the values `High` for total amounts of ₹2,000 or more and `Regular` otherwise.
18. Perform a PySpark join between registrations and the event master. Group by city and calculate the total confirmed ticket revenue.

Section D — Delta table creation and CRUD

19. Create a managed Delta table named `confirmed_event_registrations` using CTAS. Include only confirmed records.
20. Create another Delta table using a PySpark DataFrame and `saveAsTable()`.
21. Insert two new registrations into `event_registrations_work` using SQL.
22. Update registration 2003 from Pending to Confirmed.

23. Increase the unit price of all VIP registrations in the working table by ₹200.
24. Delete Cancelled registrations from the working table and verify the remaining records.

Section E — MERGE, UPSERT and SCD

25. Compare `incoming_registrations` with `event_registrations_work`. Identify existing and new registration IDs.
26. Perform a SQL `MERGE` that updates existing registrations and inserts new registrations.
27. Verify the changes to registrations 2002, 2008, and 2024.
28. Verify that registrations 2033, 2034, and 2035 were inserted.

Section F — Delta Lake internals and Time Travel

31. Execute `DESCRIBE DETAIL` on `event_registrations_work`. Identify the table format, location, number of files, and size.
32. Execute `DESCRIBE HISTORY` and identify the INSERT, UPDATE, DELETE, and MERGE operations.
33. Retrieve the table version immediately before the MERGE using `VERSION AS OF`.
34. Compare registration 2002 in the historical version with its current record.
35. Explain the purpose of Parquet files and the `_delta_log` directory in a Delta table. Identify what information is stored in each.
36. Explain what `VACUUM` does. Run `VACUUM ... DRY RUN` if supported by your environment, and identify which files would be eligible for removal without deleting them.

Section G — Managed tables and views

37. Identify whether `event_registrations_source` is a managed or external table and retrieve its storage location.
38. Create a permanent SQL view named `event_revenue_view` containing event name, city, and total confirmed revenue.
39. Create a temporary view containing only Pending registrations.
40. Create a global temporary view containing VIP registrations, if your compute supports global temporary views.
41. Query all three views and explain the difference in their scope and lifetime.

Section H — Parquet conversion and optimization

42. Execute `OPTIMIZE` on `event_registrations_work`.

43. Compare the table's file count before and after optimization using `DESCRIBE DETAIL`.
44. Run `OPTIMIZE ... ZORDER BY (event_id)` if the table does not use liquid clustering. Explain when ZORDER is useful.
45. Using a writable, approved Unity Catalog volume or external location, create a Parquet dataset from a PySpark DataFrame containing event records, then convert or load that Parquet data into a Delta table. No file upload is required.

Section I — Structured Streaming

46. Create a streaming DataFrame using Spark's built-in `rate` source, generating five records per second.
47. Display the streaming DataFrame and identify its `timestamp` and `value` columns.
48. Use `withColumn()` to generate a `transaction_id` and a `transaction_amount` from the streaming values.
49. Filter streaming records where `transaction_amount` exceeds ₹500.
50. Aggregate streaming records by a derived transaction category, display the running counts using a supported streaming sink, and stop the query.